

AGENTE TONWINI

Ecosistema Autónomo de Ingeniería del Conocimiento y Solución Académica

Documento de Arquitectura y Manual Técnico (Versión 1.0)

Desarrollado por Agustín Prunés Fuenzalida

Descripción General: Este documento detalla la arquitectura completa de pipelines, la especificación técnica de cada script y el mapeo estructural de directorios del **Agente Tonwini**. Es un sistema multi-agente guiado por IA, completamente agnóstico al dominio (capaz de procesar desde humanidades hasta ciencias exactas), diseñado para la ingesta bibliográfica híbrida, la redacción continua de libros de texto teóricos y la resolución autónoma de guías académicas utilizando $\text{\LaTeX}$.

Índice

1. Registro Audiovisual de Operación	2
2. Instalación y Configuración (Quickstart)	2
2.1. 1. Entorno Virtual y Dependencias	2
2.2. 2. Configuración de API Keys (.env)	2
3. Arquitectura de Pipelines del Sistema	4
3.1. 0. Capa Meta-Cognitiva y Orquestación General	4
3.2. 1. Pipeline de Ingesta (El Cerebro Híbrido)	4
3.3. 2. Pipeline de Libros (Producción de Teoría)	4
3.4. 3. Pipeline de Guías (Resolución Práctica Autónoma)	5
4. Mapeo Arquitectónico de Directorios	6
5. Manual de Operación Diaria	6
5.1. El Flujo Unificado (Recomendado)	6
5.2. Ejecución Manual (Modo Avanzado)	6
6. Apoyo al Proyecto y Comunidad	7

1 Registro Audiovisual de Operación

Para facilitar la comprensión del ecosistema en un entorno de ejecución real, se ha documentado un registro audiovisual que detalla el flujo de trabajo completo del Agente Tonwini. En este material se expone el proceso de configuración mediante la interfaz de hiperparámetros, la indexación bibliográfica, el razonamiento de los agentes durante la generación de contenido y la fase de auto-curación forense del código $\LaTeX$.

Visualización del sistema en ejecución: [\[Acceder a la demostración en video\]](#)

2 Instalación y Configuración (Quickstart)

Para desplegar y ejecutar el entorno del Agente Tonwini, se requiere **Python 3.10+** y una distribución funcional de $\LaTeX$ en el sistema (ej. MiKTeX o TeX Live).

2.1 1. Entorno Virtual y Dependencias

Clona el repositorio y configura un entorno virtual para aislar las dependencias:

```
git clone https://github.com/AgustinPrunes/Proyectos.git
cd Proyectos/Agente_Tonwini

# Creacion y activacion del entorno virtual
python -m venv venv
# En Windows: .\venv\Scripts\activate
# En Linux/Mac: source venv/bin/activate

# Instalacion de requerimientos
pip install -r requirements.txt
```

2.2 2. Configuración de API Keys (.env)

El agente utiliza el modelo Gemini de Google. Debes crear un archivo llamado `.env` en la raíz del proyecto y colocar tu clave API:

```
# Crea un archivo .env en la raiz e incluye esta linea:
GEMINI_API_KEY=AIZA_SyTuClaveAQUI...
```

Costos y Consumo de API (Advertencia Importante)

El Agente Tonwini está diseñado para ser exhaustivo. Utiliza llamadas iterativas masivas. Ten especial cuidado con:

- **El módulo OCR:** Procesa documentos mediante llamadas multimodales. Evalúa la cantidad y el peso de tus PDFs escaneados para no exceder cuotas.
- **El bucle Actor-Crítico:** Al resolver guías o compilar $\text{\LaTeX}$, los agentes debaten y se corrigen automáticamente, multiplicando el uso de tokens.

Asegúrate de revisar las cuotas gratuitas de Google AI Studio o establecer límites de facturación si usas una cuenta de pago.

3 Arquitectura de Pipelines del Sistema

Tonwini segmenta sus capacidades en una Capa Maestra de Orquestación y tres flujos (pipelines) secuenciales, modulares e interconectados. Cada componente ha sido diseñado con resiliencia de red y aislamiento de procesos.

3.1 0. Capa Meta-Cognitiva y Orquestación General

Actúa como el sistema nervioso central del ecosistema, unificando la ejecución de los agentes.

- **Tonwini.py:** Orquestador Maestro. Al ejecutarse, invoca de forma automática la interfaz interactiva (`Tonwini_setup.py`) para capturar los hiperparámetros del usuario (Dominio, Nivel de Rigor Académico y modo documental). Luego, presenta un menú de control con tres vías principales de ejecución automatizada:

- **Opción 1 (Libros):** Despliega el pipeline completo de Teoría.
- **Opción 2 (Guías):** Despliega el pipeline de Resolución Práctica.

Adicionalmente, intercepta caídas críticas y genera análisis forenses mediante IA en el directorio `Errores/`.

- **check_models.py y context_packer.py:** Módulos auxiliares encargados de la verificación de disponibilidad de los modelos y del empaquetado seguro del contexto recuperado.

3.2 1. Pipeline de Ingesta (El Cerebro Híbrido)

Encargado de la extracción estructural y semántica de la bibliografía de soporte. Se ejecuta en un ciclo cerrado de tres pasos:

1. **Primera pasada (build_vector_db.py):** Analiza el directorio de entrada. Indexa PDFs con texto nativo en una base de datos vectorial (ChromaDB). Los documentos escaneados son detectados y derivados a la carpeta `bibliografia_escaneada/`.
2. **Rescate Multimodal (ocr_transcriber.py):** Procesa los documentos escaneados y los transcribe a código $\text{\LaTeX}$ puro para su correcta asimilación.
3. **Segunda pasada (build_vector_db.py):** Re-evalúa el directorio, indexando ahora las transcripciones generadas por el OCR y consolidando el grafo semántico relacional (`conocimiento_grafo.json`).

3.3 2. Pipeline de Libros (Producción de Teoría)

Diseñado para la redacción ininterrumpida de textos de estudio con un rigor analítico ajustable.

- **temario_builder.py:** Expande la directriz inicial generando un programa académico adaptado al modo seleccionado (extendiéndose profundamente para un tratado exhaustivo o siendo conciso para apuntes).
- **syllabus_parser.py:** Transforma el programa bruto en un esqueleto estructural estricto en formato JSON.

- `orchestrator.py`: Interpreta el JSON y construye la arquitectura de directorios físicos y archivos base `.tex`.
- `generator_agent.py`: El agente redactor principal. Redacta atómicamente cada sección del documento mediante un filtro RAG estricto para evitar alucinaciones.
- `compilar_libro.py`, `compiler_service.py` y `llm_corrector.py`: El subsistema de autocuración. Entra en un bucle cerrado que compila iterativamente el código $\text{\LaTeX}$, captura los logs de error y aplica parches sintácticos quirúrgicos hasta garantizar un PDF estable.

3.4 3. Pipeline de Guías (Resolución Práctica Autónoma)

Entorno lógico enfocado en el desarrollo metodológico y la resolución paso a paso de problemas académicos.

- `guia_builder.py`: Diseña y estructura bancos de ejercicios y problemáticas. Se adapta al modo seleccionado en el setup para determinar la dificultad y naturaleza de la evaluación, almacenándola en `guias_in/`.
- `guide_parser.py`: Extrae los enunciados en crudo asegurando que los metadatos y requerimientos lógicos mantengan un formato JSON seguro.
- `solver_agent.py`: Implementa el bucle cognitivo Actor-Crítico. Un agente (Actor) propone la resolución matemática o discursiva, y otro agente (Crítico) audita el rigor metodológico. Las iteraciones y el debate cognitivo se registran en `Pensamientos/`.
- `compilar_guia.py`: Empaqueta el solucionario final en un documento estandarizado $\text{\LaTeX}$, ubicándolo de forma compilable en `guias_out/`.

4 Mapeo Arquitectónico de Directorios

Directorio: `bibliografia/` y `bibliografia_escaneada/`

Naturaleza: Entrada.

Contenido: Aloja los libros de texto y PDFs de base. Los escaneados se mueven automáticamente a la carpeta secundaria para procesamiento OCR.

Directorio: `guias_in/` y `guias_out/`

Naturaleza: Entrada/Salida de Práctica.

Contenido: Deposita aquí exámenes o guías en `guias_in/`. El Agente dejará las resoluciones limpias y compilables en `guias_out/`.

Directorio: `Errores/` y `Pensamientos/`

Naturaleza: Monitoreo y Trazabilidad.

Contenido: Registra `tracebacks` crudos de `crasheos` (`Errores/`) y guarda el razonamiento interno de los agentes Actor-Crítico al resolver problemas (`Pensamientos/`).

Directorio: `chroma_db/` (**Autogenerada**)

Naturaleza: Base de Datos Local.

Contenido: Almacena los `embeddings` vectoriales persistentes de tu bibliografía.

5 Manual de Operación Diaria

El diseño de la Capa Meta-Cognitiva simplifica drásticamente el uso del ecosistema. Mediante el orquestador unificado, se delega al software la evaluación del estado de los directorios y la activación secuencial de los pipelines.

5.1 El Flujo Unificado (Recomendado)

```
python Tonwini.py
```

Al ejecutar `Tonwini.py`, el sistema llamará internamente al panel de hiperparámetros (`Tonwini_setup.py`) y luego te ofrecerá el menú operativo para enrutar el flujo hacia la construcción teórica, la resolución algorítmica o el procesamiento bibliográfico, ejecutando en cascada todos los scripts internos correspondientes.

5.2 Ejecución Manual (Modo Avanzado)

Para operaciones quirúrgicas o diagnóstico de componentes individuales, el entorno permite invocar cada módulo de manera aislada:

```
# Indexar Base de Datos Vectorial (Ciclo Hibrido Completo)
python build_vector_db.py
python ocr_transcriber.py
python build_vector_db.py
```

```
# Ejecucion segmentada del Pipeline de Libros
Luego de escribir los temas en temario_bruto.txt
python temario_builder.py
python syllabus_parser.py
python orchestrator.py
python generator_agent.py
python compilar_libro.py

# Ejecucion segmentada del Pipeline de Guias
Luego de escribir los temas en temas_guia_txt
python guia_builder.py
python guide_parser.py
python solver_agent.py
python compilar_guia.py
```

6 Apoyo al Proyecto y Comunidad

El Agente Tonwini es un proyecto de código abierto (Open Source) desarrollado con el objetivo de facilitar el acceso a herramientas avanzadas de Inteligencia Artificial para la educación y la ingeniería del conocimiento.

Si este ecosistema te ha sido de utilidad para tus estudios, proyectos o flujos de trabajo, considera apoyar el desarrollo continuo. Tu contribución ayuda a mantener el código actualizado, potenciar futuras funcionalidades y financiar horas de investigación.

Invítame un café o apoya el proyecto: [\[Apoyar en Ko-fi\]](#)